

CEN4025C Module 4 Project

Securing the TaskManager Frontend with Route Guards and Interceptors

In Module 3, we connected the TaskManager Angular frontend to the Spring Boot backend using HttpClient and implemented REST-based communication. In this module, we will extend the existing TaskManagerFrontEnd project by implementing:

- Authentication (login/logout)
- Route Guards
- HTTP Interceptors

The goal is to introduce frontend-level security patterns commonly used in production Angular applications.

The following changes have been applied to the TaskManagerFrontEnd project:

1. Added a simple mock login mechanism
2. Protected selected routes using an Angular Route Guard
3. Attached authentication headers to outgoing requests using an HTTP Interceptor

1. Clone the repo and examine the new and modified files:

src/app/services/auth.service.ts

- Handles login, logout, token storage, and authentication state.

src/app/guards/auth.guard.ts

- Implements CanActivate
- Redirects unauthenticated users to /login.

src/app/interceptors/auth.interceptor.ts

- Adds Authorization: Bearer <token> header
- Logs outgoing HTTP requests.

src/app/components/login/login.ts

src/app/components/login/login.html

src/app/components/login/login.css

- Login form
- Calls AuthService
- Redirects after successful login

src/app/components/subtasks-page/subtasks-page.ts

src/app/components/subtasks-page/subtasks-page.html

src/app/components/subtasks-page/subtasks-page.css

- Provides a route-level component for /subtasks
- Separate from the embedded subtasks component used inside Tasks

src/app/app.routes.ts

- Added routes:

- /login
 - /tasks (protected)
 - /users (protected)
 - /subtasks (protected)
- Applied canActivate: [authGuard]

src/app/app.config.ts

- Registered interceptor using:
- provideHttpClient(withInterceptors([authInterceptor]))

src/app/app.ts

- Removed direct component embedding
- Uses <router-outlet>
- Displays login/logout/nav conditionally

src/app/app.html

- Added:
 - Navigation bar
 - Login/Logout controls
 - Conditional route links
 - <router-outlet>

2. Build and Run the Backend

Follow the same backend execution steps from Module 3:

- taskmanager_db exists
- run the Spring Boot backend in IntelliJ
- verify it runs successfully
- confirm that localhost:4200 is allowed in CorsConfig

3. Build and Run the Frontend

In the TaskManagerFrontEnd folder:

- npm install
- npm run build (or ng build)
- npm start (or ng serve)

Connect to the browser.

(NOTE: Check the Deliverables section, below, for screenshots that you will need to submit before completing the following steps).

4. Verify route guards (protected routes)

In the browser **before logging in**, try going directly to any protected route by typing it in the address bar:

- http://localhost:4200/tasks
- http://localhost:4200/users

- <http://localhost:4200/subtasks>

Expected result:

- You should be redirected to **/login** each time.

5. Log in (mock)

1. Go to <http://localhost:4200/login>
2. Enter any username/password (mock login accepts anything).
3. Click **Login**.

Expected result:

- You should be considered “logged in”
- Navigation links (Tasks/Users/Subtasks) should become visible
- A **Logout** button (and username) should appear

6. Verify protected pages load *after* login

After logging in, click:

- Tasks
- Users
- Subtasks

Expected result:

- Each page loads data from the backend.

7. Verify the HTTP interceptor (Authorization header + logging)

1. Open **DevTools** (F12) → **Console**
2. Refresh a protected page (like **Tasks**) to trigger HTTP requests.
3. You should see console logs from the interceptor for outgoing requests.

Confirm the header:

1. Open DevTools → Network
2. Click a request to the backend (e.g., /tasks)
3. In Request Headers (scroll down if necessary), look for:
Authorization: Bearer ...

Expected result:

- When logged in, outgoing requests include the Authorization header.
- When logged out, they should not.

NOTE: If real authentication were implemented, you would see a long encoded JWT token in the Authorization: Bearer <token> header instead of Bearer mock-token, and that token would be validated by the backend on each request.

8. Verify logout behavior

1. Click **Logout**
2. Try visiting:
 - <http://localhost:4200/tasks>

Expected result:

- Redirect back to /login
- Protected links disappear again

Deliverables

1. Push your updated project to the GitHub Classroom repository.
2. Provide screenshots showing:
 - Redirect when accessing protected route without authentication
 - Tasks page after authentication
 - Browser console log showing logged requests
 - Browser Network log showing authorization header